

Programovací jazyky a překladače

Přednáška 10, poznámky

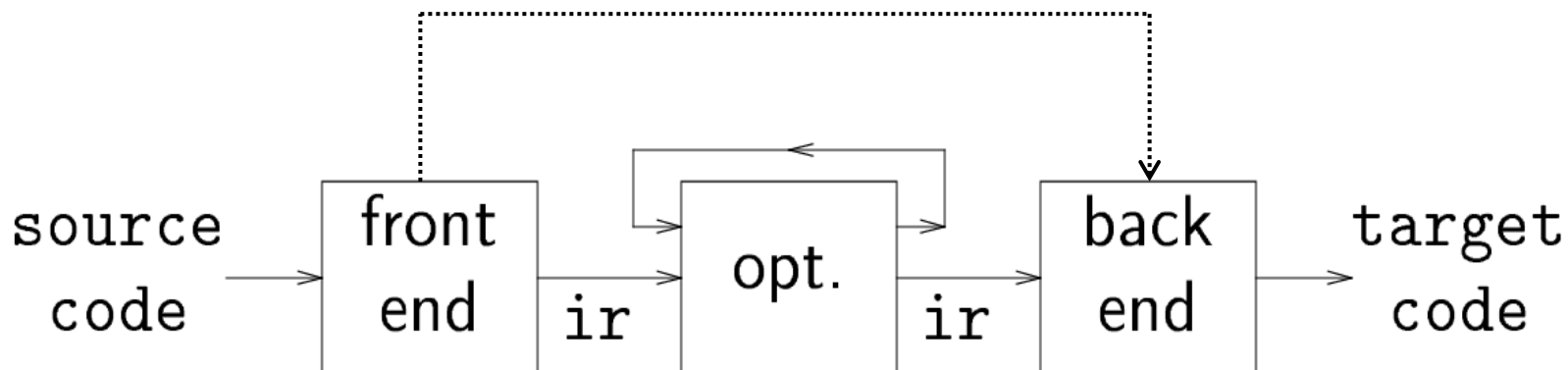
Jan Voráček

VŠPJ Jihlava, voracek@vspj.cz

Dnešní témata

- Prezentace
 - Typová kontrola
 - Vlastnosti programu za běhu
 - Tabulka symbolů
- Generování mezikódu
- Tabulka symbolů
- Struktura programu za běhu (runtime)

Vnitřní forma (mezikód /MK/, intermediální kód)



- Může, ale nemusí existovat
- Může mít jednu nebo více forem (viz následující strany)
- Neplatí zde striktní pravidla, ale volba MK často souvisí s cílovou platformou

Proč se jím zabývat:

- Dvoustupňový vývoj je jednodušší
- Možnost podpory různých cílových platforem
- Uplatnění strojově nezávislých optimalizací

Formální reprezentace mezikódu

1. **Strukturované** (grafy, stromy):

- Akce generují odpovídající grafy stromového typu
- **Abstract Syntax Tree** (AST) nebo optimalizovaný Directed Acyclic Graph (DAG)
 - Mechanizmy řízení jsou řešeny specifickými formami průchodu grafem

2. **Nestrukturované, využívající zásobníku**

- **Implementace: Postfix**, P-code, Java bytecode, MS CIL (Common IM Lang.)
- Akce pracují s vrcholem zásobníku (viz následující slajdy)
 - Řízení (statements) je realizováno přesuny na příslušné kompaktní registrové bloky (expressions, basic blocks)

3. **Nestrukturované , využívající *n-tic* (registrové formy)**

- **Implementace: Tříadresový kód, čtveřice** apod.
- Akce využívají registry (viz následující slajdy)
 - Řízení je implementováno příslušnými skokovými instrukcemi

Rozdělení procesorů

- **Virtuální:** libovolná architektura (SW)
- **Reálné:**
 - CISC (Complex Instruction Set Computer)
 - RISC (Reduced Instruction Set Computer) – LOAD, STORE
- **Registrové:** mají více registrů, což podporuje bohatější instrukční sadu (překládaný program vede na celkově méně instrukcí), ale vyžaduje jistý čas na dekodování instrukce
- **Zásobníkové:** pracují pouze s vrcholem zásobníku, což vede na jednodušší adresaci, ale obvykle vyžaduje více instrukcí
 - JVM VM: zásobníkový, interpret
 - Dalvik VM (Google): registrový , JIT překladač
 - ART VM: registrový, AOT (od A v.7.0 AOT+JIT)

Registrový a zásobníkový stroj, srovnání

- Registrový (virtuální) procesor:

R1: 5				R1: 5
R2: 10	→	ADD R1, R2, R3	→	R2: 10
R3: 50				R3: 15
R4: 100				R4: 100

- Zásobníkový (virtuální) procesor:

SP: 5		POP 5		SP: 15
10		POP 10		20
20	→	ADD 5, 10, result	→	100
100		PUSH result		

Zásobníkový a registrový procesor, ilustrativní příklad

Instrukce: **B + C - D**

Postfix: B C + D -

Zásobníkový kód:

```
push val B
push val C
add
push val D
sub
```

Instrukce: **A = B**

Postfix: A B =

Zásobníkový kód:

```
push add A
push val B
store
```

B + C - D

```
load r0, B
load r1, C
add r0, r1, r0
load r2, D
sub r0, r2, r0
```

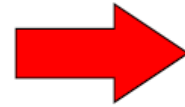
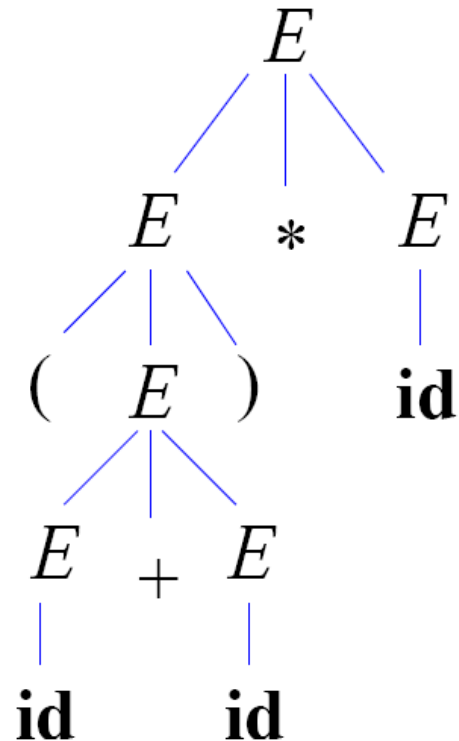
A = B

```
load r0, add A
load r1, val B
store r1, (r0)
```

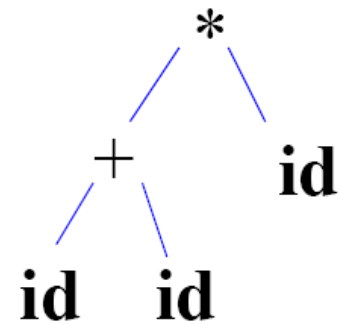
1. Strukturovaná reprezentace

(a + b) * c

Parse Tree



AST



SDT pro AST aritmetických výrazů, S-atributová LR gramatika

$E \rightarrow E1 + T \quad \{ E.\text{node} = \text{Node}(' + ', E1.\text{node}, T.\text{node}); \}$

$E \rightarrow T \quad \{ E.\text{node} = T.\text{node}; \}$

$T \rightarrow (E) \quad \{ T.\text{node} = E.\text{node}; \}$

$T \rightarrow \text{id} \quad \{ T.\text{node} = \text{Leaf}(\text{id}, \text{id}.\text{entry}); \}$

SDT pro AST aritmetických výrazů, L-atributová LL gramatika

$E \rightarrow T A$ { $E.\text{node} = A.s$;
 $A.i = T.\text{node}$; }

$A \rightarrow + T A1$ { $A1.i = \text{Node}(' + ', A.i, T.\text{node})$;
 $A.s = A1.s$; }

$A \rightarrow \epsilon$ { $A.s = A.i$; }

$T \rightarrow (E)$ { $T.\text{node} = E.\text{node}$; }

$T \rightarrow \text{id}$ { $T.\text{node} = \text{Leaf}(\text{id}, \text{id.entry})$; }

Konstrukce syntaktického stromu, *yacc*

S	→	Expr	$$$ = \$1;$
Expr	→	Expr + Term	$$$ = \text{MakeAddNode}(\$1, \$3);$
		Expr - Term	$$$ = \text{MakeSubNode}(\$1, \$3);$
		Term	$$$ = \$1;$
Term	→	Term * Factor	$$$ = \text{MakeMulNode}(\$1, \$3);$
		Term / Factor	$$$ = \text{MakeDivNode}(\$1, \$3);$
		Factor	$$$ = \$1;$
Factor	→	(Expr)	$$$ = \$1;$
		num	$$$ = \text{MakeNumNode}(\text{token});$
		id	$$$ = \text{MakeIdNode}(\text{token});$

2. Třídresový kód (TAC)

(vhodný pro registrové stroje)

- Definovaná sada instrukcí, realizujících činnosti na úrovni výrazů a příkazů 
 - Aritmetické a logické operace, přiřazení, indexace, adresace, řízení běhu (skoky, návěští), volání funkcí a návraty
- Instrukce má vždy pouze jeden operátor s jednoznačnou aritou (počtem operandů) a nemá nikdy více jak 3 operandy:

$x := y \text{ op } z$

kde x, y, z mohou být jména, konstanty nebo dočasné proměnné překladače. Výraz typu:

$d := x + y * z$

je pak přeložen jako :

$t1 := y * z$

$t2 := x + t1$

$d := t2$

Příklady instrukcí TAC

Operace a práce s pamětí

$x := y \text{ op } z$ přiřazení (uložení)

$x := \text{op } y$ unární přiřazení

$x := y$ kopírování

Skoky

`goto L` nepodmíněný skok

`if x relop goto L` podmíněný skok

Procedury/funkce

`param x`

`call p n`

`return y`

Pole

$x := y[i]$

$x[i] := y$

Ukazatele

$x := \&y$

$x := *y$

$*x = y$

Implementace příkazů TAC: čtveřice

$a := b * -c + b * -c$

#	Op	Arg1	Arg2	Res
(0)	uminus	c		t1
(1)	*	b	t1	t2
(2)	uminus	c		t3
(3)	*	b	t3	t4
(4)	+	t2	t4	t5
(5)	:=	t5		a

Implementace příkazů TAC: trojice

$a := b * -c + b * -c$

#	Op	Arg1	Arg2
(0)	uminus	c	
(1)	*	b	(0)
(2)	uminus	c	
(3)	*	b	(2)
(4)	+	(1)	(3)
(5)	:=	a	(4)

Implementace příkazů TAC: nepřímé trojice

$a := b * -c + b * -c$

#	Stmt	
(0)	(14)	-->
(1)	(15)	-->
(2)	(16)	-->
(3)	(17)	-->
(4)	(18)	-->
(5)	(19)	-->

#	Op	Arg1	Arg2
(14)	uminus	c	
(15)	*	b	(14)
(16)	uminus	c	
(17)	*	b	(16)
(18)	+	(15)	(17)
(19)	:=	a	(18)

Program

Zásobník trojic

Generování mezikódu, syntaxí řízené definice (SDD)

Atributy pro výrazy (*expressions*) E :

E.place : **adresa** paměťového místa s hodnotou E

E.code : instrukce, vyhodnocující E

Viz následující slajd



Atributy pro příkazy (*statements*) S :

S.begin : **adresa** první instrukce kódu, odpovídajícímu S

S.after : **adresa** první instrukce kódu, následujícímu po S

S.code : instrukční soubor, odpovídající S

Existence více adres činí zpracování příkazů problematické

TAC, SDD

S -> id:=E	S.code := E.code gen(id.place ':=' E.place)	Výstup: ASS(E.place, - , id.place)
E -> E1+E2	E.place := newtemp; E.code := E1.code E2.code gen(E.place ':=' E1.place '+' E2.place)	Výstup: ADD(E1.place, E2.place, E.place)
E -> E1*E2	E.place := newtemp; E.code := E1.code E2.code gen(E.place ':=' E1.place '*' E2.place)	Výstup: MUL(E1.place, E2.place, E.place)
E -> -E1	E.place := newtemp; E.code := E1.code gen(E.place ':=' 'uminus' E1.place)	Výstup: UMINUS(E1.place, - ,E.place)
E -> (E1)	E.place := E1.place; E.code := E1.code;	
E -> id	E.place := id.place; E.code := ""	

Problematika příkazů a logických výrazů

Zdrojový kód

if $x + 2 > 3 * (y - 1) + 4$ **then** $z := 0$;

Odpovídající mezikód:

```
t1 := x+2
```

```
t2 := y-1
```

```
t3 := 3*t2
```

```
t4 := t3+4
```

```
if t1 <= t4 goto L
```

```
z := 0
```

```
L:  ...
```

Přímočarý způsob práce s logickými výrazy

Syntaktické pravidlo: **BExp -->E1 relop E2**

Odpovídající mezikód:

```
            t1 <-- hodnota E1
            t2 <-- hodnota E2
        t3 := TRUE
        if t1 relop t2 goto L
        t3 := FALSE
    label L: ...
```

Nedostatky:

- Velké množství pomocných proměnných
- Častý přístup do paměti
- Nemožnost zkráceného vyhodnocování výrazů

Podmíněné příkazy

Syntaktické pravidlo:

S --> if E then S1 else S2

Sémantická pravidla:

```
{      S.begin := newlabel();
      S.after  := newlabel();
      S.code   := gen('label' S.begin)      |
      E.code   |
      gen('if' E.place '=' ' 0' ' goto' S2.begin) |
      S1.code  |
      gen('goto' S.after) |
      S2.code  |
      gen('label' S.after)
}
```

Cykly

Syntaktické pravidlo: **S --> while E do S1**

Struktura generovaného kódu:

```
L1 :    Vyhodnoť E
      if (E == FALSE) goto L2
      Kód pro S1
      goto L1
L2 :    ...
```

Sémantická pravidla:

```
{    S.begin := newlabel();
    S.after  := newlabel();
    S.code  := gen('label' S.begin)
        E.code
        gen('if' E.place '=' ' 0' 'goto' S2.after)
        S1.code
        gen('goto' S.begin)
        gen('label' S.after)
}
```

Generování adres pomocí markerů

E --> E1 or M E2

- Marker nemá žádnou syntaktickou akci, tj. $M \rightarrow \varepsilon$

Zdrojový kód: **E = a<b or c<d and e<f**

Odpovídající mezikód:

```
100 : if a < b goto __E_true__  
101 : goto 102  
102 : if c < d goto 104  
103 : goto __E_false__  
104 : if e < f goto __E_true__  
105 : goto __E_false__
```

E.truelist = {100; 104}

E.falselist = {103; 105}

E.true : instrukce

E.false: instrukce

Přímočará (intuitivní, naivní) adresace

Vede na nepodmíněné skoky a ne_monotonní adresaci:

Zdrojový kód: **while a < b do**
 if x < y then S endif
 endwhile

Odpovídající mezikód:

```
L1 :   if a < b goto L2
      if x < y goto L3
      goto L4
L3 :   S.code
L4 :   goto L1
L2 :
```

Tento nedostatek lze opět řešit pomocí markerů na úrovni gramatického pravidla (*backpatching*)

S --> if E then M1 S1 N else M2 S2
S --> while M1 E do M2 S1

3. Postfixová reprezentace

(vhodná pro zásobníkové stroje)

- **Infix**

- Způsob, jakým běžně zapisujeme aritmetické výrazy:

$$a+b*(c^d-e)^{(f+g*h)}-i$$

- **Postfix**

- Způsob, vhodný pro interpretaci aritmetického výrazu zásobníkovým strojem:

$$abcd^e-fgh*+^*+i-$$

Naplnění zásobníku formou postfixové reprezentace

Gramatika:

$E \rightarrow E + T \{ \text{push } + \}$

$E \rightarrow E - T \{ \text{push } - \}$

$E \rightarrow T$

$T \rightarrow T * F \{ \text{push } * \}$

$T \rightarrow T / F \{ \text{push } / \}$

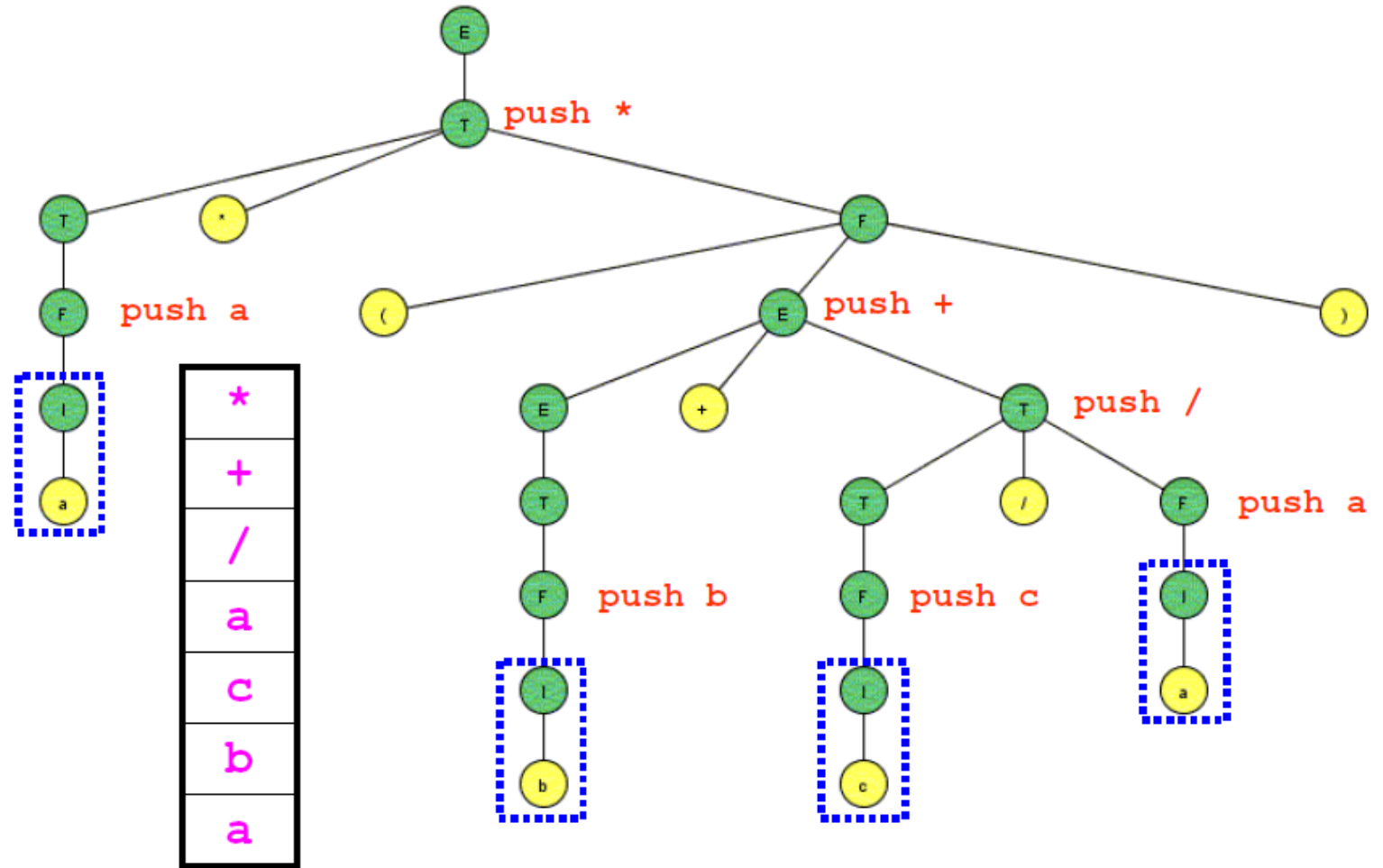
$T \rightarrow F$

$F \rightarrow i \{ \text{push } i \}$

$F \rightarrow (E)$

Vstup:

$a*(b+c/a)$



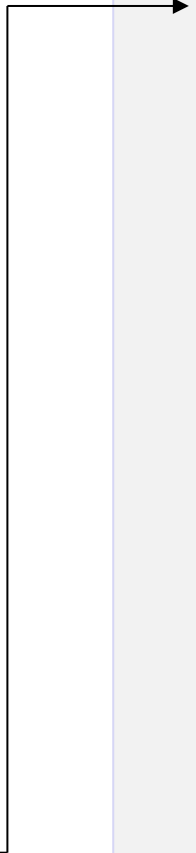
Generování cílového kódu z postfixu

Vstup: $a = a * (b + c / a)$

Výstup:

```
push dword a
push dword [a]
push dword [b]
push dword [c]
push dword [a]
pop eax
pop ebx
idiv ebx, eax
push ebx
```

```
pop eax
pop ebx
add ebx, eax
push ebx
pop eax
pop ebx
mul ebx, eax
push ebx
pop ebx
pop eax
mov dword [eax], ebx
```



Mezikód, příklad č. 1 – aritmetický výraz

Za předpokladu platnosti standardních pravidel pro asociativitu a prioritu operátorů převeďte výraz:

$$a^{*-(b+c)}$$

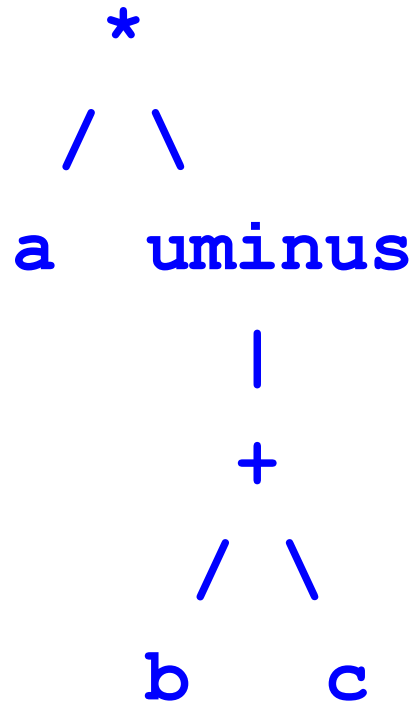
do následujících forem mezikódu:

- a) Abstraktní syntaktický strom
- b) Postfixová notace
- c) Tříadresový kód

Poznámka: MK pro aritmetický výraz reprezentuje nejjednodušší případ, tzv. základní blok

Mezikód, příklad č. 1, řešení

AST:



$a * -(b + c)$

Postfix:

$a \ b \ c \ + \ uminus \ *$ *nebo*
 $b \ c \ + \ uminus \ a \ *$

TAC:

$t1 = b + c$
 $t2 = - \ t1$
 $t3 = a * t2$

Mezikód, příklad č. 2 – logický výraz

Za předpokladu platnosti standardních pravidel pro asociativitu a prioritu operátorů převeďte výraz:

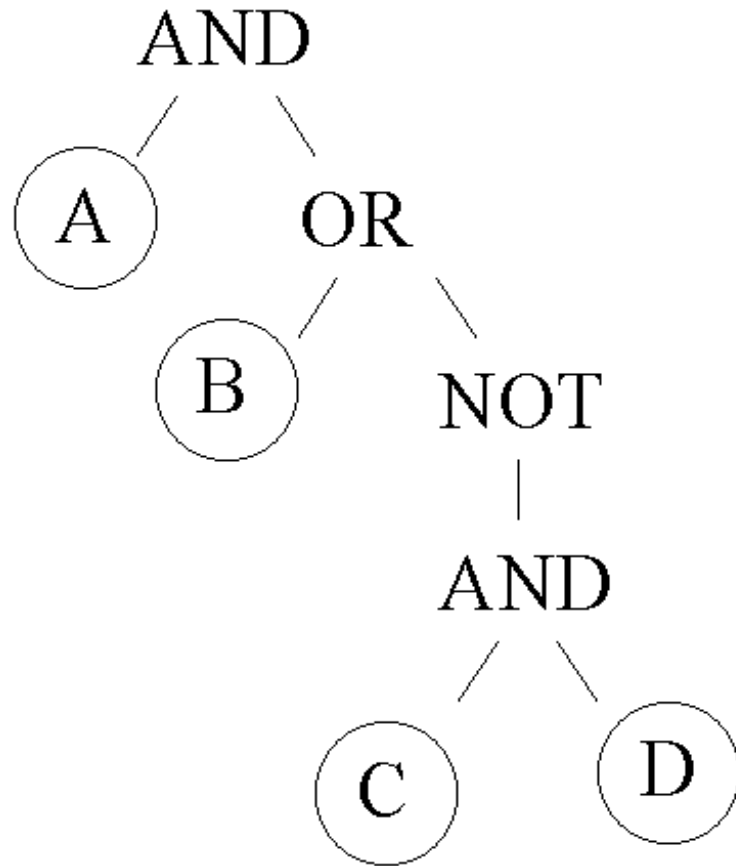
A and (B or not (C and D))

do následujících forem mezikódu:

- Abstraktní syntaktický strom (AST nebo DAG)
- Tříadresový kód v libovolné formě reprezentace

Mezikód, příklad č. 2, řešení

1) AST:



`A and (B or not (C and D))`

2) TAC

`T1 := C and D`

`T2 := not T1`

`T3 := B or T2`

`T4 := A and T3`

TAC, zkrácená forma

`if not A goto FALSE`

`if B goto TRUE`

`if not C goto TRUE`

`if D goto FALSE`

`TRUE:`

`...`

`FALSE:`

`...`

Mezikód, příklad č. 3, řídicí příkaz

Převeďte následující příkaz do TAC:

```
if (a < b + c)
```

```
    a = a - c;
```

```
c = b * c;
```


Mezikód, příklad č. 3, řešení

```
if (a < b + c)
    a = a - c;
c = b * c;
```



```
t1 = b + c;
t2 = a < t1;
FJP L0;
t3 = a - c;
a = t3;
L0: t4 = b * c;
c = t4;
```

Generování vnitřní reprezentace programu

Miroslav Beneš
Dušan Kolář



Tabulka symbolů, shrnutí

- **Jméno** a na něj v různých fázích navázané následující atributy:

- **Adresa a hodnota**

- Vazba adresy a hodnoty může být statická nebo dynamická

- **Typ**

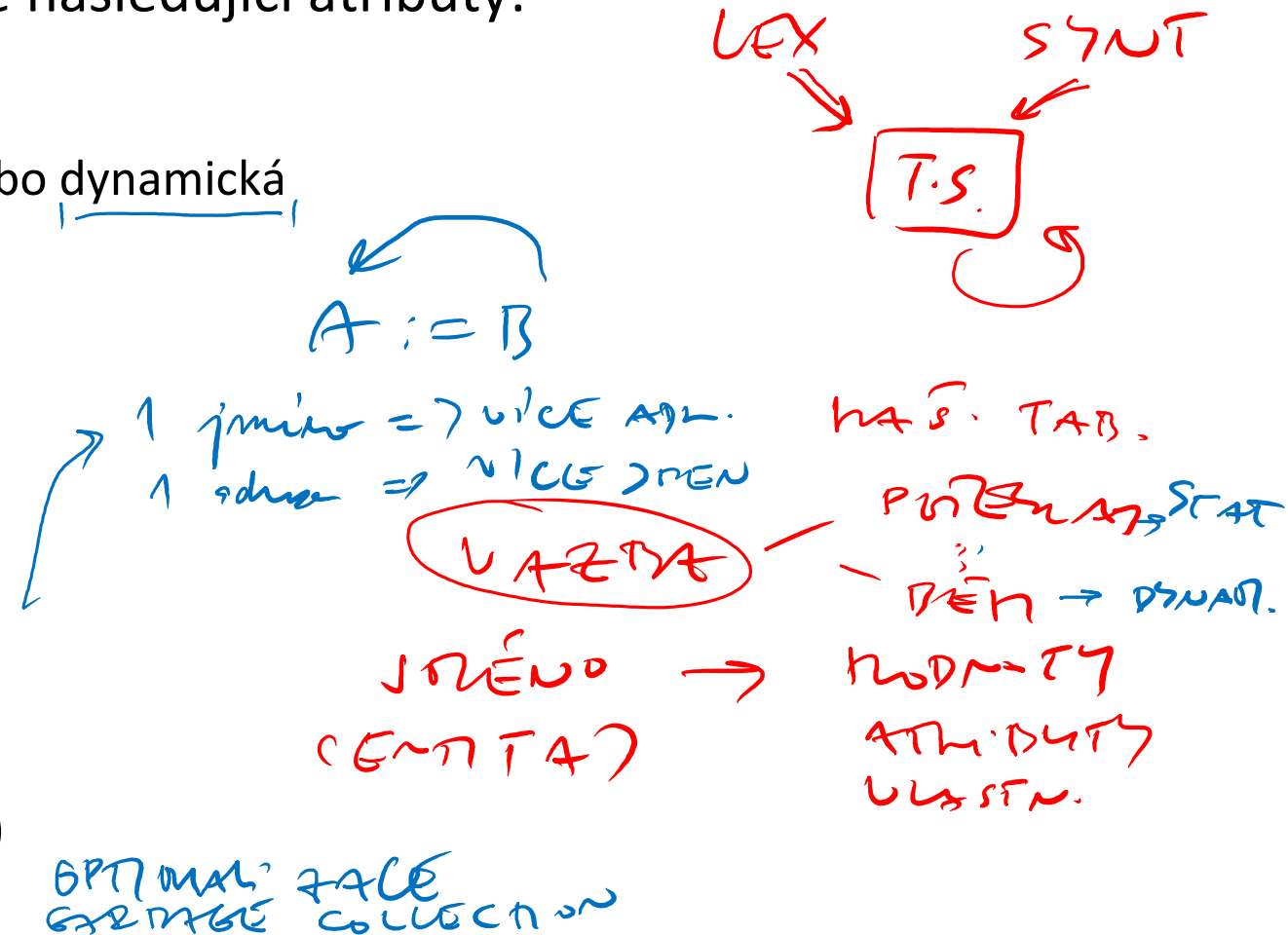
- Definuje paměťový rozsah daného jména
- **Statické typy**: rozsah známý v čase překladu
- Dynamické typy: rozsah známý za běhu

- **Rozsah platnosti** (viditelnost proměnné)

- **Statický rozsah**: známý v čase překladu
- Dynamický rozsah: známý za běhu

- **Doba života** (existence přidělené paměti)

- Statická alokace: za překladu
- Dynamická alokace: při vstupu do rozsahu platnosti (automatické proměnné), nebo za běhu (new)



Statický a dynamický rozsah platnosti (scope)

```
#include<stdio.h>
int x = 10;

int f()
{
    return x;
}

int g()
{
    int x = 20;
    return f();
}

int main()
{
    printf("%d", g());
    printf("\n");
    return 0;
}
```

Výstup: 10

```
// Jde o demonstrační pseudokód
int x = 10;

int f()
{
    return x;
}

int g()
{
    int x = 20;
    return f();
}

main()
{
    printf(g());
}
```

Výstup: 20

Nejjednodušší varianta TS: vrstvený zásobník

```
void scopes ()  
{ int a, b, c;  
  ...  
  { int a, b;  
    ...  
  }  
  { float c, d;  
    { int m;  
      ...  
    }  
  }  
}
```



c	int
b	int
a	int

b	int
a	int
c	int
b	int
a	int

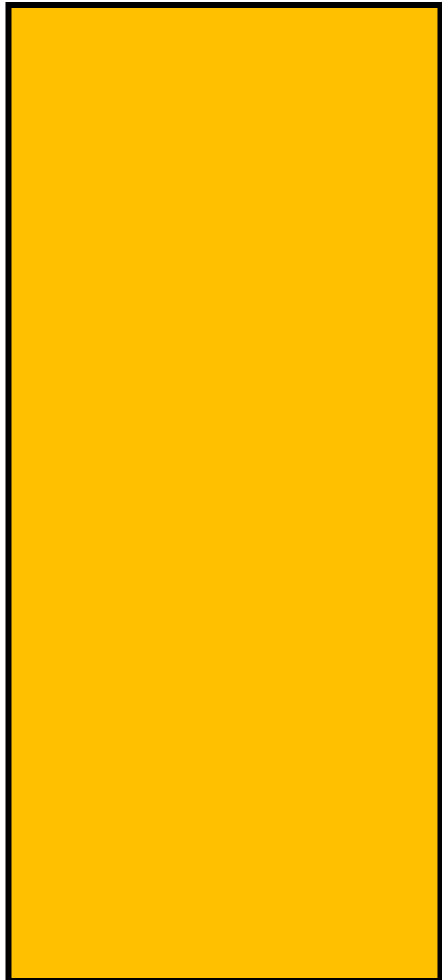
c	int
b	int
a	int

d	float
c	float
c	int
b	int
a	int

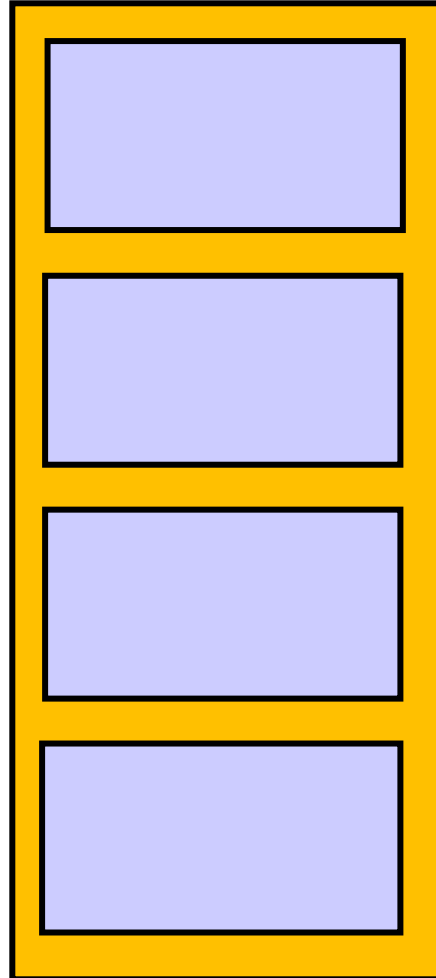
m	int
d	float
c	float
c	int
b	int
a	int

Různé typy blokové struktury tabulky symbolů

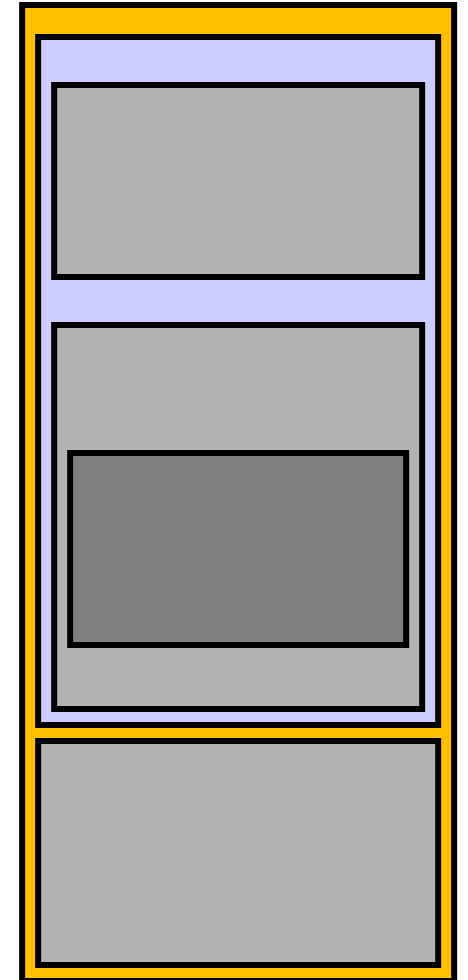
Monolitická



Jednoduchá bloková



Vnořená bloková



Monolitická struktura

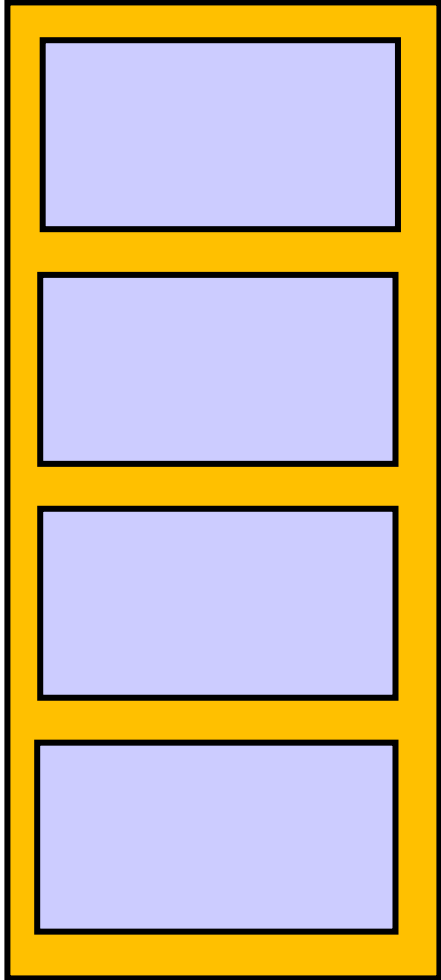
Charakteristika

- Celý program je reprezentován jediným blokem
- Každý identifikátor je přístupný odkudkoli

Rozsah:

- Žádný identifikátor nemůže být deklarován více než jednou
- Každému výskytu identifikátoru předchází odpovídající deklarace

Jednoduchá bloková struktura



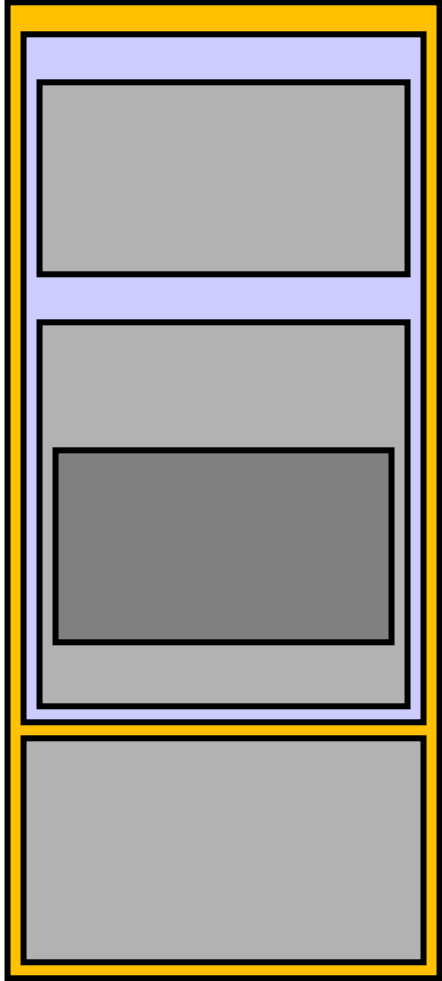
Charakteristika

- Celý program může být rozdělen do nezávislých disjunktních bloků

Rozsah:

- Existují pouze lokální a globální proměnné
 - Globální identifikátor může být lokálně předefinován
 - Stejný identifikátor se může objevit v různých blocích
 - Každému výskytu identifikátoru předchází odpovídající lokální nebo globální deklarace
-

Vnořená bloková struktura



Charakteristika

- Umožňuje definovat blok uvnitř bloku
- Typická pro většinu moderních programovacích jazyků
- Vzhledem je strukturní složitosti existuje celá řada implementačních forem
 - Pole, seznam, strom, hašovací tabulka či kombinace těchto struktur

Rozsah:

- Stejný identifikátor se nemůže objevit v vícekrát v jednom bloku
- Každý identifikátor musí být deklarován buď v témže, nebo souvisejícím nadřazeném bloku

Tabulka symbolů, příklad

Mějme následující programový fragment jazyka C :

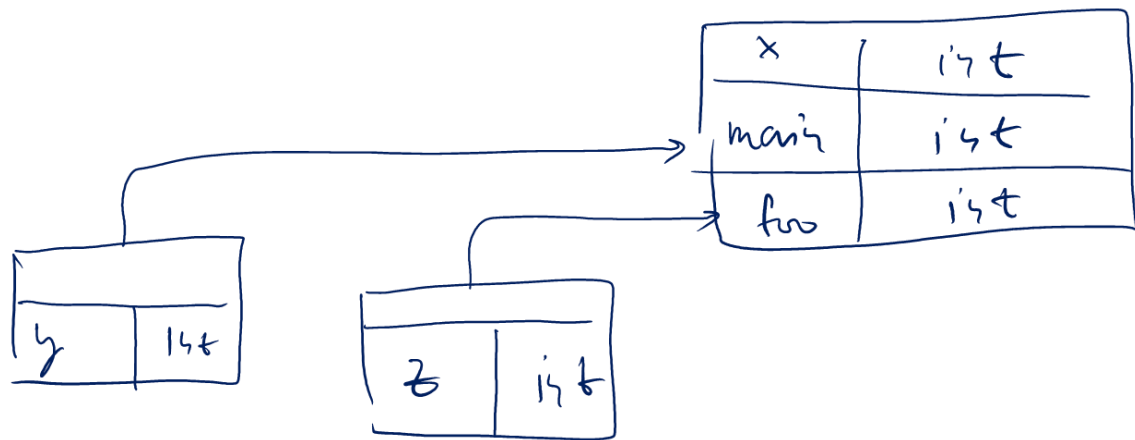
```
int x;  
int main( ) { int y; ... }  
int foo( ) { int z; ... // *NOW* }
```

Jaké bude struktura odpovídající tabulky symbolů v čase překladu?

Zakreslete její reálnou situaci v okamžiku *NOW*

Tabulka symbolů, řešení

```
int x;  
int main( ) { int y; ... }  
int foo( ) { int z; ... // *NOW* }
```



```
...  
int value=10;  
  
void pro_one()  
{  
    int one_1;  
    int one_2;  
  
    {  
        int one_3;  
        int one_4;  
    }  
  
    int one_5;  
  
    {  
        int one_6;  
        int one_7;  
    }  
}  
  
void pro_two()  
{  
    int two_1;  
    int two_2;  
  
    {  
        int two_3;  
        int two_4;  
    }  
  
    int two_5;  
}  
...
```

Symbol	Type	Scope
pro_one	proc	global
pro_two	proc	global

Symbol	Type	Scope
one_1	int	proc para
one_2	int	proc para
one_5	int	Proc para

Symbol	Type	Scope
two_1	int	proc para
two_2	int	proc para
two_5	int	proc para

Symbol	Type	Scope
one_3	int	inner
one_4	int	inner

Symbol	Type	Scope
one_6	int	inner
one_7	int	inner

Symbol	Type	Scope
two_3	int	inner
two_4	int	inner

Podobný příklad viz https://www.tutorialspoint.com/compiler_design/compiler_design_symbol_table.htm

Tabulka symbolů

Miroslav Beneš

Dušan Kolář



Struktura programu za běhu

Akce zdrojového programu, zajišťující bezchybný běh programu po jeho spuštění

- Řízení běhu programu
 - Iniclace, finalizace, chyby, komunikace s OS
- Reprezentace dat v paměti
- Přidělování paměti
 - Časové i objemové
- Spouštění podprogramů (parametry, návratové hodnoty)

Fáze překladu

- Vstupy:
 - Zdrojový kód + odpovídající header_y, knihovny atd.
- Výstupy:
 - Assembler, relokovatelný objekt nebo spustitelný soubor
 - Informační výstupy překladače (varování, chyby)
- Chyby detekovatelné v překladu
 - Syntaxe
 - Statické datové typy a funkční prototypy
- Úspěšný překlad znamená, že program:
 - Má formální smysl v daném programovacím jazyce
 - Je možno ho spustit

Fáze běhu programu (runtime)

- Vstupy a výstupy
 - Závisí plně na programátorovi
- Chyby za běhu
 - Dělení nulou
 - Čtení z nealokované paměti
 - Nedostatek paměti, přetečení zásobníku
 - Přístup k neexistujícímu souboru (FD)
- Úspěšné spuštění znamená, že:
 - Program běží nebo skončí očekávaným způsobem

Vztah mezi analýzou (překladem) a syntézou (vykonáním)

Kód: **statická část funkce**

- Typové a rozsahové náležitosti se uchovávají v **tabulce symbolů**

Aktivace: **dynamická část (vykonání) funkce**

- Paměťové, datové a řídicí náležitosti za běhu jsou součástí tzv. aktivačního záznamu
 - Aktivační záznam (activation record) je při volání uložen do runtime zásobníku při volání a zase je z něj uvolněn při ukončení
 - Graficky ho lze znázornit **aktivačním stromem**

Analýza zdrojového kódu

Syntéza cílového kódu

Deklarace

Alokace

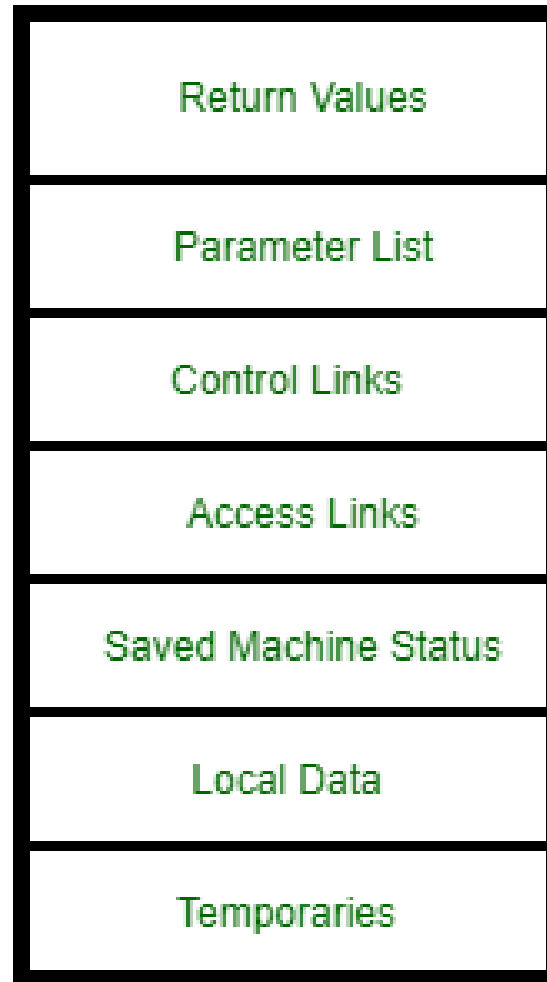
Procedura

Aktivace

Rozsah

Doba života

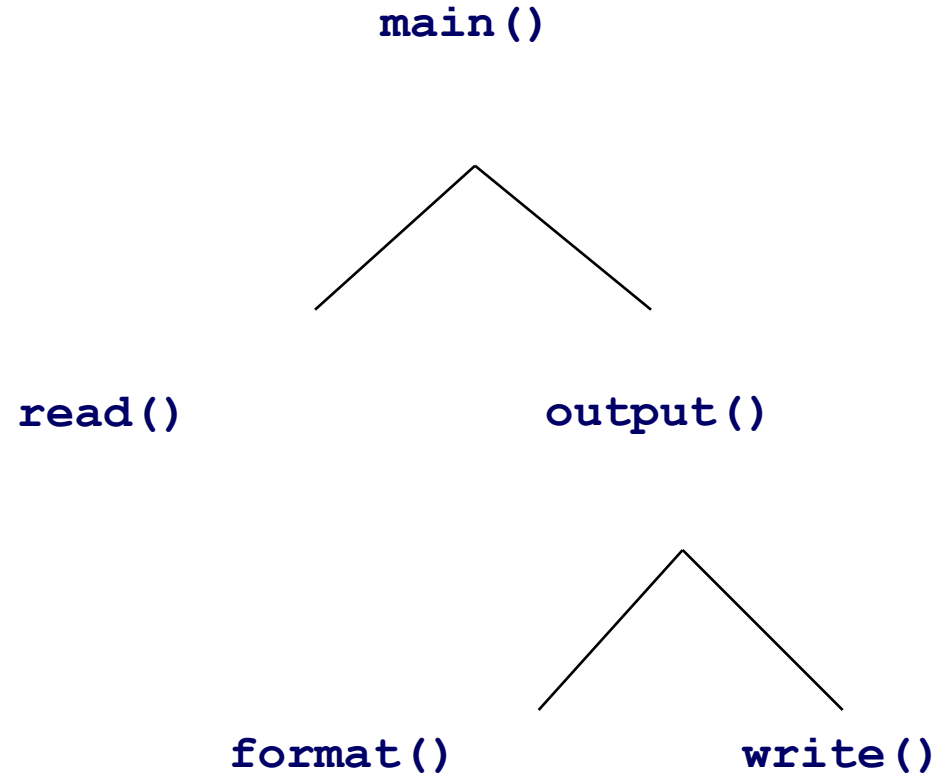
Struktura aktivačního záznamu



Zdroj: <https://www.geeksforgeeks.org/access-links-and-control-links/>

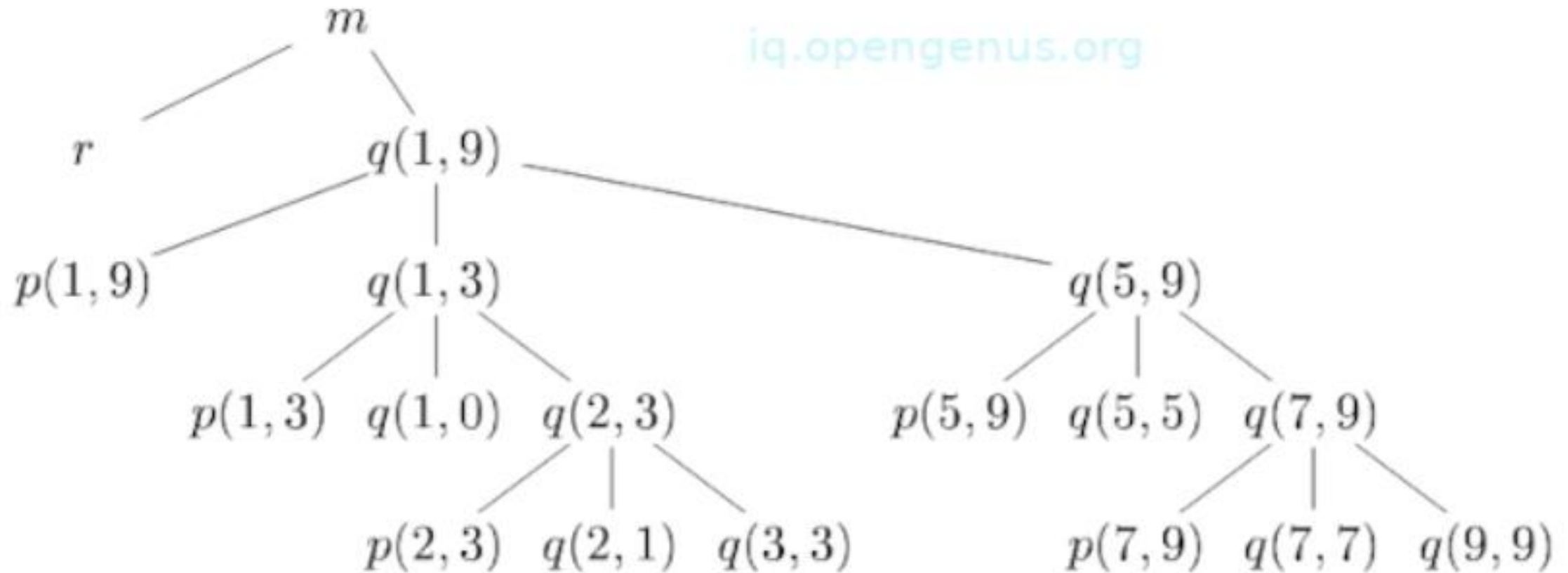
Příklad aktivačního záznamu a odpovídajícího stromu

```
main()  
{  
  read();  
  output();  
}  
  
output()  
{  
  format();  
  write();  
}
```



Quicksort (1,9), příklad aktivačního stromu

iq.opengenus.org



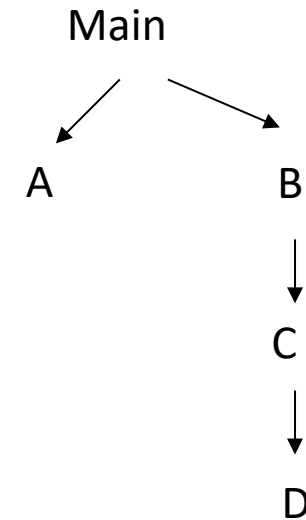
m: hlavní funkce (main)
r: načtení datového pole (read)

p: partition (hledání pivoa, např. $p(1,9) = 4$, $p(1,3) = 1$, $p(2,3) = 2$, --, $p(7,9) = 8$ atd.

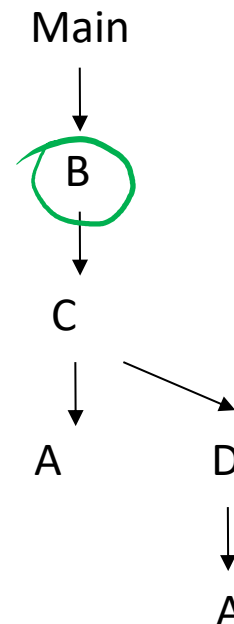
Lexikální a run-time rozsah (vnořené funkce)

```
program Main (input, output);  
  var x, y, z: integer;  
  → procedure A;  
    var x: integer; begin { A }  
      x := 1; y := x * 2 + 1  
    end;  
  → procedure B;  
    var y: real;  
    procedure C;  
      var z: real;  
      procedure D;  
        var y: real; begin { D }  
          x := 1.25 * z; A; writeln('x = ', x)  
        end;  
      begin { C }  
        z := 1; A; D;  
      end;  
    begin { B }  
      C; writeln('x = ', x)  
    end;  
  begin { Main }  
    x := 0; B;  
  end.
```

Lexikální rozsah
(rozsah při překladu, **statický**)

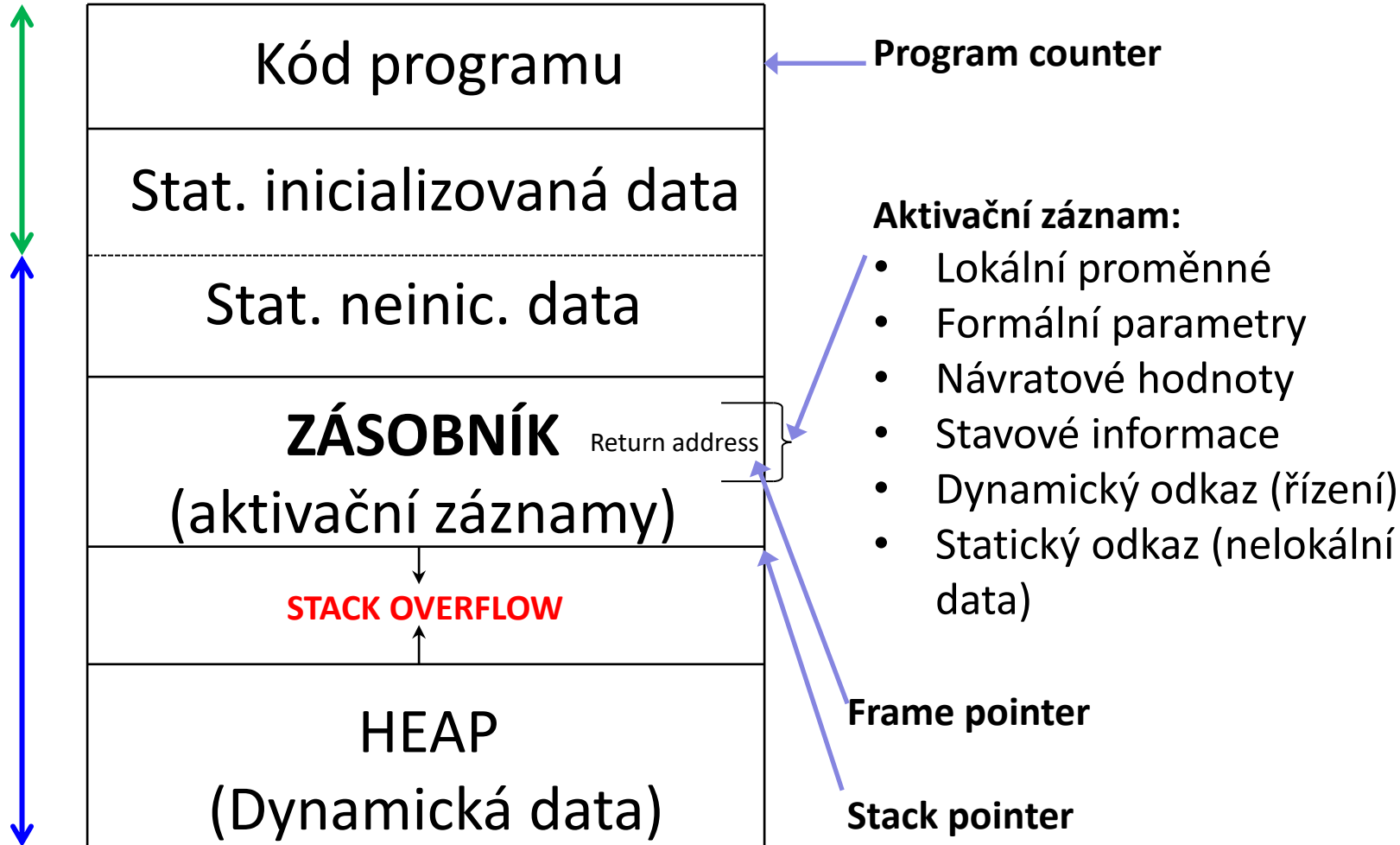


Aktivační strom
(rozsah za běhu, **dynamický**)



Organizace paměti za běhu

Uloženo ve exe souboru



Alokováno za běhu

```
begin                --lex level 1
  real v1;            --level 1 obj 1
  real v2;            --level 1 obj 2
  proc A =            --level 1 obj 3
    begin            --lex level 2
      real v3;        --level 2 obj 1
      proc B =        --level 2 obj 2
        begin        --lex level 3
          real v4;     --level 3 obj 1
          ...
        end{B};
      B               --call B
    end{A};
  end{A};

  proc C =            --level 1 obj 4
    begin            --lex level 2
      real v5;        --level 2 obj 1
      proc D =        --level 2 obj 2
        begin        --lex level 3
          real v6;     --level 3 obj 1
          A            --call A
        end{D};
      D               --call D
    end{C};
  end{C};

  C                  --call C
end
```

--LA, CS UWA

Příklad :

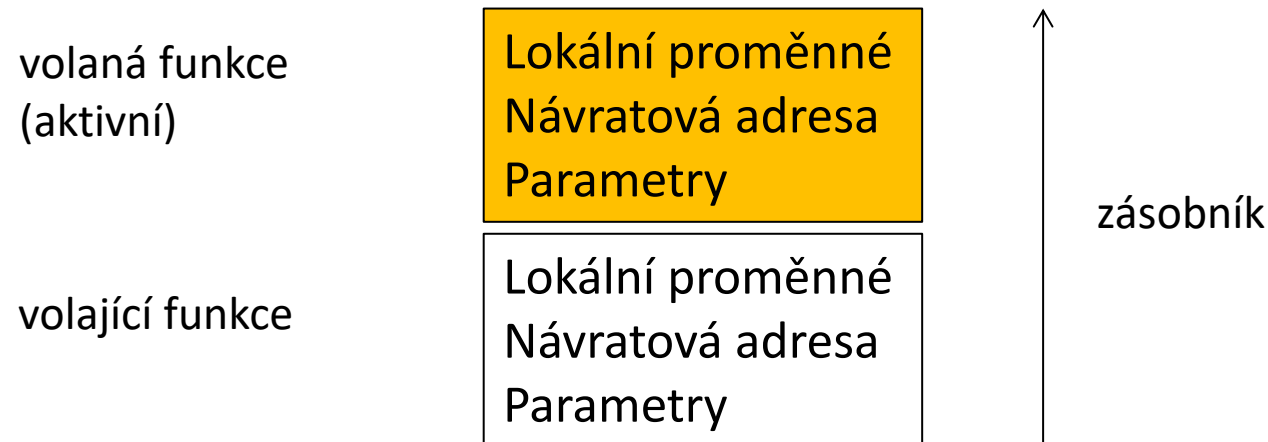
<http://www.allisons.org/II/ProgLang/PL-Block/>

Volání funkce

- Volající funkce:
 - *param t*: vyhodnotí aktuální parametry *t* a předá je volané funkci
 - *call p, n*: uloží aktuální stavové informace (ukazatele zásobníků a návratovou adresu) a předá řízení volané funkci *p* společně s počtem parametrů *n*
- Volaná funkce:
 - *enter m*: uloží registry a aktualizuje ukazatele zásobníků vzhledem k hodnotě *m*

Návrat z funkce

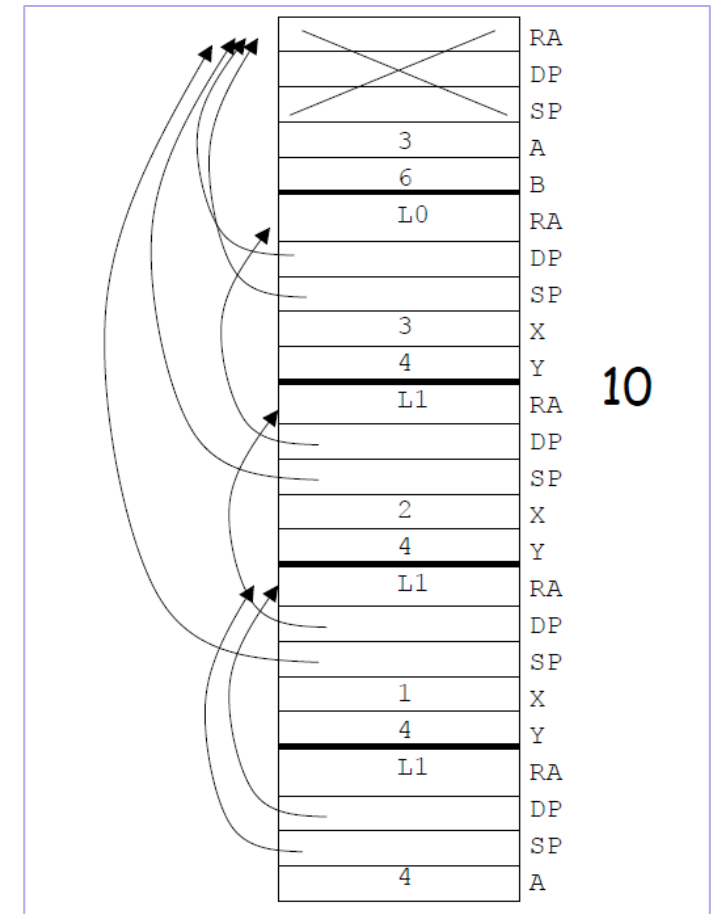
- Volaná funkce:
 - *return x* nebo *return*: existuje-li, vrátí volané funkci hodnotu *x*, aktualizuje ukazatele zásobníků a přejde na návratovou adresu.
- Volající funkce:
 - *retrieve x*: existuje-li, uloží hodnotu *x*, vrácenou volanou funkcí



Aktivační záznam, příklad + řešení

Mějme následující kód a předpokládejme, že parametry jsou předávány hodnotou. Nakreslete situaci na zásobníku za běhu programu (runtime stack) bezprostředně po umístění aktivačního záznamu pro proceduru *two*. Co bude vytištěno na konci programu?

```
program main();
  var A,B: integer;
  procedure one(X,Y: integer);
    procedure two(A: integer);
    begin
      B := B + A;
    end;
  begin
    if (X = 1) then two(Y);
    else begin B:=B+1; one(X-1,Y); end;
  end;
begin
  A := 3; B := A+1;
  one(A,B);
  write(B);
end.
```



Struktura programu v době běhu

Miroslav Beneš
Dušan Kolář

